

Bluetooth Fundamentals

Bluetooth Fundamentals

1. Bluetooth Overview
 - 1.1 What is Bluetooth
 - 1.2 Bluetooth Version Evolution
 - 1.3 Bluetooth Frequency Bands and Channels
 - 1.4 Bluetooth vs WiFi
 2. Bluetooth Protocol Stack Architecture
 - 2.1 Dual-Mode Architecture
 - 2.2 Host and Controller Layers
 - 2.3 Layer Details
 3. Classic Bluetooth vs Bluetooth Low Energy
 - 3.1 Core Differences
 - 3.2 Why BLE Has Low Power Consumption
 4. BLE Core Concepts
 - 4.1 GAP — Generic Access Profile
 - Four GAP Device Roles
 - Advertising PDU Types
 - Advertising Data Format
 - 4.2 ATT — Attribute Protocol
 - Attribute Structure
 - GATT Operations and Permission Mapping
 - 4.3 GATT — Generic Attribute Profile
 - UUID Structure
 - CCCD — Client Characteristic Configuration Descriptor
 - 4.4 Notify vs Indicate Comparison
 - 4.5 BLE Connection Parameters
 5. BLE Advertising and Connection Process
 - 5.1 Complete Lifecycle
 - 5.2 Advertising Interval and Discovery Latency
 - 5.3 BLE 5.0 Extended Advertising
 6. SM — Security Manager
 - 6.1 Pairing Methods
 - 6.2 Three Phases of Pairing
 7. Classic Bluetooth Profiles Introduction
 8. ESP32-S3 Bluetooth Hardware Features
 - 8.1 Bluetooth and WiFi Coexistence — 2.4GHz Resource Arbitration
 - 8.2 Coexistence Configuration
 9. ESP-IDF Bluetooth Protocol Stack Selection
 - 9.1 Bluedroid vs NimBLE
 - 9.2 Protocol Stack Selection
 - 9.3 Bluedroid menuconfig Key Configuration
 10. Common Troubleshooting
-

1. Bluetooth Overview

1.1 What is Bluetooth

Bluetooth is a short-range wireless communication technology operating in the **2.4 GHz ISM band** (2400~2483.5 MHz) for data exchange between devices over short distances. The name originates from the 10th-century Nordic King Harald Bluetooth (Harald Blåtann), symbolizing the unification of communication between different devices.

Key Parameter	Description
Frequency Band	2.4000 ~ 2.4835 GHz (ISM unlicensed band)
Modulation	GFSK (Gaussian Frequency Shift Keying), BLE 1M / 2M Physical Layer
Frequency Hopping	Adaptive Frequency Hopping (AFH), 1600 hops/s (Classic Bluetooth) / Pseudorandom hopping (BLE)
Communication Range	Typical 10m (Class 2, 0~4 dBm), Max 100m (Class 1, +20 dBm)
Network Topology	Point-to-Point (Piconet) / Broadcast / Mesh (BLE Mesh)

The Bluetooth name comes from Danish King Harald "Bluetooth" Gormsson, who unified the tribes of the Scandinavian Peninsula - just as Bluetooth technology aims to unify short-range communication protocols between different devices.

1.2 Bluetooth Version Evolution

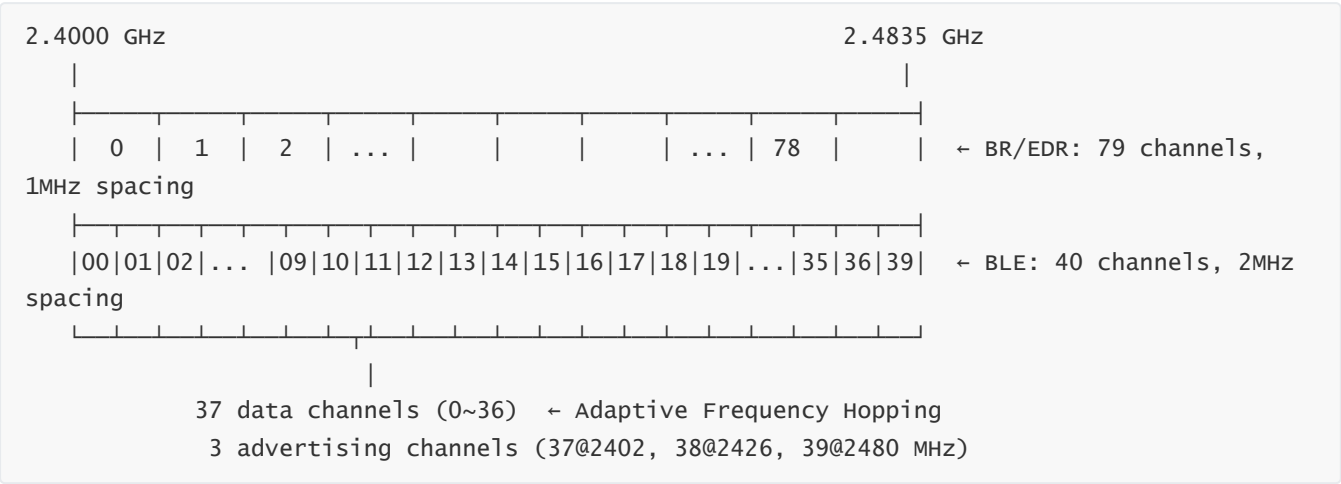
Version	Year	Key Features	Max Speed
1.0 / 1.0B	1999	First commercial version, serious interoperability issues	721 kbps
1.1 / 1.2	2001~2003	AFH adaptive frequency hopping, reduced WiFi interference	721 kbps
2.0 + EDR	2004	Enhanced Data Rate (EDR), $\pi/4$ -DQPSK / 8DPSK modulation	2.1 Mbps
2.1 + EDR	2007	Secure Simple Pairing (SSP), reduced pairing complexity	2.1 Mbps
3.0 + HS	2009	Introduced 802.11 (WiFi) as high-speed channel	24 Mbps
4.0	2010	BLE first introduced , low power mode, coin battery can last months	1 Mbps (BLE)
4.1 / 4.2	2013~2014	BLE enhancement: LE Secure Connections, Data Length Extension	1 Mbps

Version	Year	Key Features	Max Speed
5.0	2016	2M PHY, LE Long Range (Coded PHY), Advertising Extensions	2 Mbps
5.1	2019	AoA / AoD direction finding (centimeter-level)	2 Mbps
5.2	2020	LE Audio, LC3 codec, Isochronous Channels	2 Mbps
5.3 / 5.4	2021~2023	Channel classification enhancement, Periodic advertising enhancement, PAwR	2 Mbps

ESP32-S3 supports **BLE 5.0** and **Classic Bluetooth (BR/EDR)**, theoretical max speed: ~1400 kbps application layer throughput with BLE 2M PHY.

1.3 Bluetooth Frequency Bands and Channels

The 2.4GHz ISM band is divided into different channel grids based on Bluetooth type:



Bluetooth Type	Number of Channels	Channel Spacing	Hopping Rate
BR/EDR (Classic Bluetooth)	79	1 MHz	1600 hop/s
BLE (Bluetooth Low Energy)	40 (37 data + 3 advertising)	2 MHz	Adaptive pseudorandom

BLE places advertising channels deliberately at 2402 / 2426 / 2480 MHz - these three frequencies avoid the most commonly used WiFi channels 1, 6, 11 (2412 / 2437 / 2462 MHz), reducing mutual interference with WiFi at the physical layer.

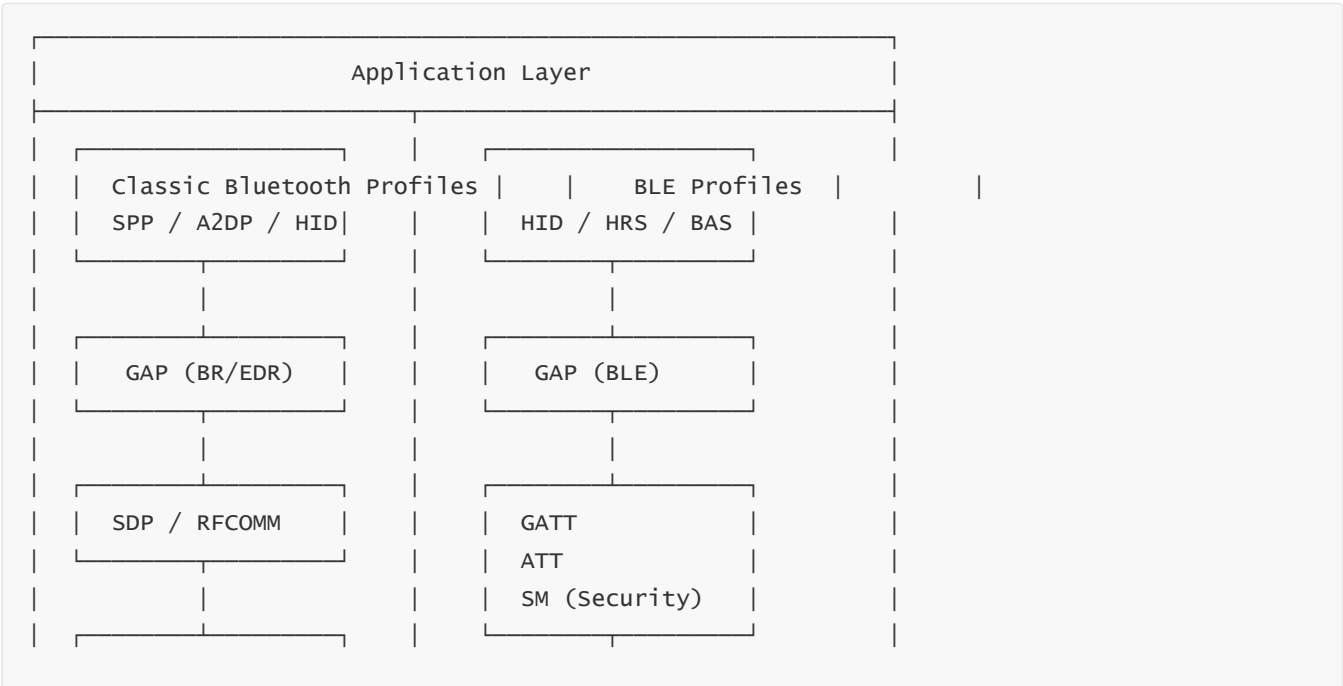
1.4 Bluetooth vs WiFi

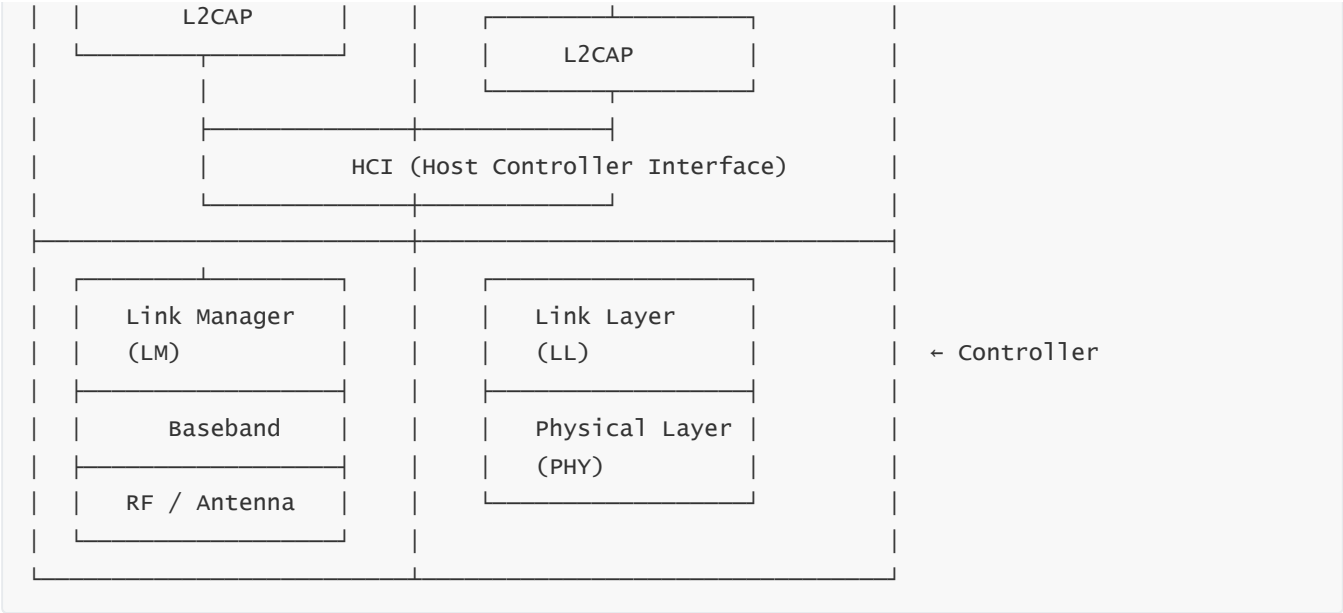
Comparison	Bluetooth (BLE 5.0)	WiFi (802.11n)
Frequency Band	2.4 GHz	2.4 GHz / 5 GHz
Channel Bandwidth	2 MHz	20 / 40 MHz
Max Application Layer Speed	~1.4 Mbps	~50 Mbps (ESP32-S3)
Power Consumption	Extremely low (µA level sleep)	Relatively high (mA level continuous reception)
Connection Establishment	3 ms	100 ms ~ several seconds
Typical Communication Range	10~100m	30~100m
Network Topology	Point-to-Point / Broadcast / Mesh	Star (AP + STA)
Use Cases	Wearables, sensors, audio	Internet, video streaming, OTA
IP Protocol Stack	None (requires upper layer encapsulation like 6LoWPAN)	Native TCP/IP

2. Bluetooth Protocol Stack Architecture

2.1 Dual-Mode Architecture

ESP32-S3 supports **dual-mode Bluetooth** - the same chip can run Classic Bluetooth (BR/EDR) and Bluetooth Low Energy (BLE) simultaneously, both sharing the RF front-end but with independent protocol stacks:





2.2 Host and Controller Layers

The Bluetooth protocol stack is divided into two major parts, communicating through **HCI (Host Controller Interface)**:

Layer	Sub-modules	Responsibilities
Host	GAP / GATT / ATT / SM / L2CAP / SDP / RFCOMM	Protocol logic, security management, service discovery
HCI	UART / USB / SDIO transport	Standard communication interface between Host and Controller
Controller	Link Layer / Baseband / PHY / RF	RF control, baseband processing, frequency hopping, physical layer modulation/demodulation

On ESP32-S3, Host and Controller run on the same chip, with HCI communicating through an internal virtual interface (not physical UART). On phones and other devices, Host typically runs on the main CPU with Controller as a separate Bluetooth chip.

2.3 Layer Details

Layer	Full Name	Function	Key Concepts
PHY	Physical Layer	Radio transceiver, GFSK modulation	1M / 2M / Coded PHY
LL	Link Layer	Advertising, scanning, connection establishment/maintenance, hopping	Advertising state / Scanning state / Connection state
HCI	Host Controller Interface	Command/event channel between Host and Controller	CMD / Event / ACL Data

Layer	Full Name	Function	Key Concepts
L2CAP	Logical Link Control & Adaptation Protocol	Data fragmentation/reassembly, protocol multiplexing	CID (Channel ID), MTU
SM	Security Manager	Pairing, bonding, encryption key distribution	Just Works / Passkey / OOB
ATT	Attribute Protocol	Read/write/notify protocol for attribute data	Attribute Handle / UUID / Permissions
GATT	Generic Attribute Profile	Data organization model	Service → Characteristic → Descriptor
GAP	Generic Access Profile	Device discovery and connection management	Advertising / Scanning / Connection / Role definition

3. Classic Bluetooth vs Bluetooth Low Energy

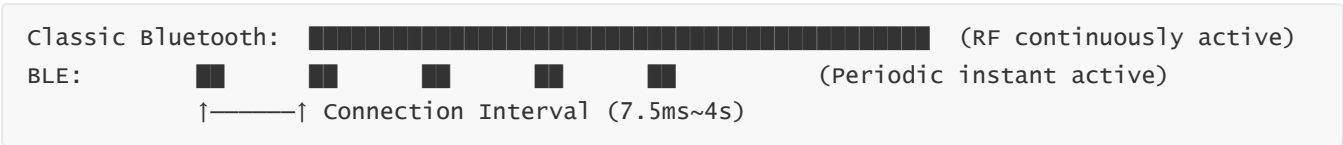
3.1 Core Differences

Feature	Classic Bluetooth (BR/EDR)	Bluetooth Low Energy (BLE)
Physical Channels	79 × 1MHz	40 × 2MHz
Hopping Rate	1600 hop/s	Adaptive (can change channels per connection event)
Connection Establishment	Scanning + Paging, ~100ms	Advertising → Scanning → Connection, ~3ms
Max Application Layer Throughput	2.1 Mbps	~1.4 Mbps (2M PHY)
Power Consumption	Medium (~1W active)	Extremely low (~0.01~0.5W, coin battery lasts months)
Connection Count	Max 7 slaves/piconet	No hard limit (depends on protocol stack implementation)
Data Model	Serial emulation (SPP/RFCOMM)	Attribute Protocol (ATT/GATT)
Typical Applications	Audio streaming, file transfer, Bluetooth headsets	Wearables, sensors, beacons, HID peripherals
Advertising Data	Limited (inquiry response)	Rich (up to 31 bytes legacy / 1650 bytes extended advertising BLE 5.0)

3.2 Why BLE Has Low Power Consumption

BLE's low power consumption comes from three design choices:

Design Choice	Classic Bluetooth	BLE	Power Impact
Connection Duty Cycle	Continuous polling	Connection interval min 7.5ms, max 4s	↓ 99% RF transmission time
Packet Length	Max 2745 bits	Legacy 376 bits / Extended 2120 bits	↓ Transmission duration
Connection Latency	None	Can skip N connection events (Slave Latency)	↓ Slave response count
Advertising	Continuous	Configurable interval 20ms ~ 10.24s	↓ Advertising RF duty cycle



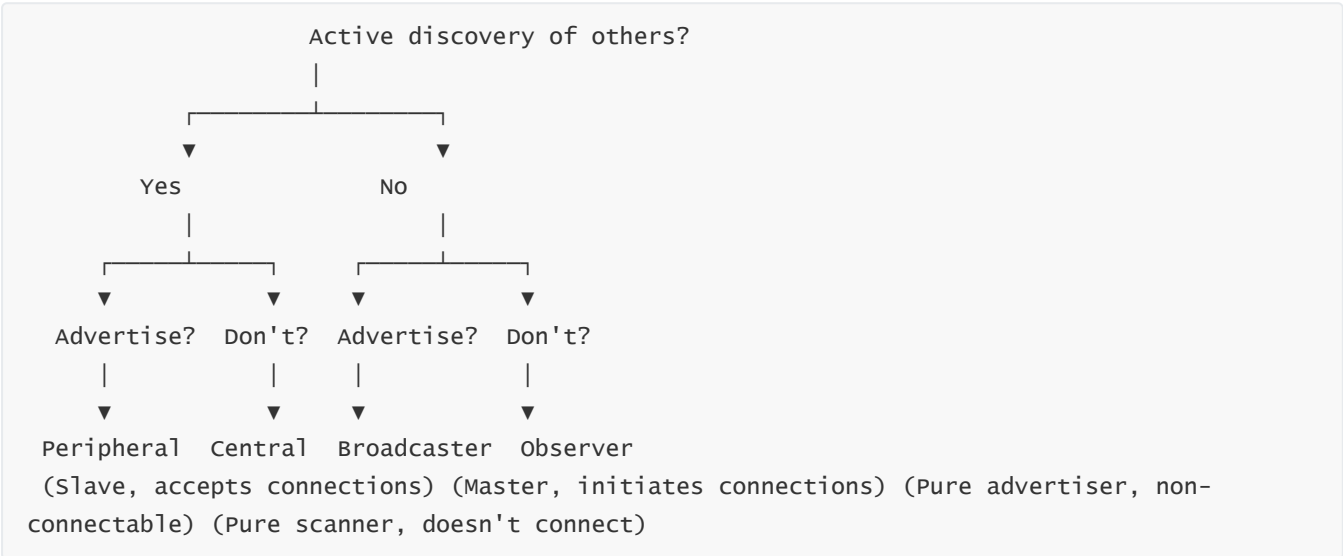
BLE's RF is in **Sleep mode** most of the time, only waking up for a few milliseconds during scheduled connection events for data transmission. This is the fundamental reason why its power consumption is only 1%~5% of Classic Bluetooth.

4. BLE Core Concepts

4.1 GAP — Generic Access Profile

GAP defines how BLE devices are **discovered and connected by other devices**, and is the entry point for all BLE communication.

Four GAP Device Roles



GAP Role	English	Sends Advertising	Initiates Connection	Typical Devices
Peripheral	Slave/Peripheral	✓	✗	Sensors, fitness bands, beacons
Central	Master/Central	✗	✓	Phones, tablets, gateways
Broadcaster	Pure broadcaster	✓	✗	iBeacon, Eddystone beacons
Observer	Pure observer	✗	✗	Passive listeners, packet sniffers

Advertising PDU Types

Advertising PDU Type	Meaning	Connectable	Scannable
ADV_IND	General advertising indication	✓ Connectable	✓ Scannable
ADV_DIRECT_IND	Directed advertising (specified target address)	✓ Connectable	✗
ADV_NONCONN_IND	Non-connectable advertising (pure beacon)	✗	✗
ADV_SCAN_IND	Scannable advertising (no connection but can receive more data)	✗	✓

Advertising Data Format

Advertising data is organized in **Length-Type-Value (LTV)** structure:

```

Advertising packet (max 31 bytes / legacy mode)
├─ [L=2] [T=Flags] [V=0x06] ← 3 bytes
├─ [L=9] [T=Complete Name] [V="ESP-IDF"] ← 11 bytes
├─ [L=3] [T=16-bit Service] [V=0xFFFF0] ← 5 bytes
└─ [L=...] other fields...
```

AD Type	Value	Description
0x01	Flags	Discoverable mode + BR/EDR support flags
0x02 / 0x03	Incomplete / Complete 16-bit Service UUIDs	Service UUID list
0x08 / 0x09	Shortened / Complete Local Name	Device name

AD Type	Value	Description
0x0A	TX Power Level	Transmit power (dBm), used for distance estimation
0xFF	Manufacturer Specific Data	Manufacturer-specific data (used by iBeacon)

4.2 ATT — Attribute Protocol

ATT (Attribute Protocol) is the foundational protocol for BLE data transmission. All data on BLE devices is stored in the form of **Attributes (Attribute)**, where each attribute is a data unit with a handle and UUID.

Attribute Structure

Attribute Handle	UUID	Permissions	Value	
0x0010	0x2800	R	0xFFFF0	← Service declaration
0x0011	0x2803	R	NOTIFY, 0xFFFF1	← Characteristic declaration
0x0012	0xFFF1	R N	"Hello"	← Characteristic value
0x0013	0x2902	R W	0x0001	← CCCD descriptor

Each attribute is defined by four elements:

Element	Description	Example
Attribute Handle	16-bit unique number, assigned by protocol stack	0x0010
Attribute Type (UUID)	Type identifier of the attribute	0x2800 = GATT Primary Service declaration
Permissions	Access permissions (read/write/notify/encrypt)	Read + Notify
Value	Attribute value (fixed or variable length)	"Hello World"

GATT Operations and Permission Mapping

Operation (ATT Layer)	GATT Method	Direction	Declaration Required
Read Request / Response	<code>read_cb()</code>	Client ← Server	<code>BLE_GATT_CHR_F_READ</code>
Write Request / Response	<code>write_cb()</code>	Client → Server	<code>BLE_GATT_CHR_F_WRITE</code>
Write Command (no response)	<code>write_cb()</code>	Client → Server	<code>BLE_GATT_CHR_F_WRITE_NO_RSP</code>

Operation (ATT Layer)	GATT Method	Direction	Declaration Required
Handle Value Notification	<code>notify()</code>	Server → Client	<code>BLE_GATT_CHR_F_NOTIFY</code>
Handle Value Indication	<code>indicate()</code>	Server → Client (requires confirmation)	<code>BLE_GATT_CHR_F_INDICATE</code>

4.3 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) is built on top of ATT and defines the **hierarchical organization model** for BLE data:

```
Profile (Specification/Application layer, e.g. Heart Rate Profile)
├── Service (UUID 16/32/128-bit)
│   ├── e.g.: 0x180D = Heart Rate Service (SIG standard)
│   ├── 0xFFFF0 = Custom service
│   └── Characteristic (The unit that actually exposes data)
│       ├── UUID + property declarations (Read/Write/Notify/Indicate)
│       ├── e.g.: 0x2A37 = Heart Rate Measurement (notification-oriented)
│       ├── 0xFFFF1 = Custom notification characteristic
│       └── Descriptor (Metadata for characteristic, optional)
│           ├── e.g.: 0x2902 = CCCD (Client Characteristic Configuration Descriptor)
│           └── 0x2901 = Characteristic User Description (human-readable text for characteristic name)
```

UUID Structure

UUID Type	Length	Format	Example
16-bit UUID (SIG Standard)	2 bytes	<code>0xxxxx</code>	<code>0x180D</code> (Heart Rate Service)
32-bit UUID	4 bytes	<code>0xxxxxxxxx</code>	<code>0x12345678</code>
128-bit UUID (Custom)	16 bytes	<code>xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx</code>	<code>4fafc201-1fb5-459e-8fcc-c5c9c331914b</code>

16-bit UUIDs are standardized identifiers assigned by the Bluetooth SIG Alliance and require purchase authorization. Custom projects can freely use 128-bit UUIDs, or use 16-bit UUIDs in non-reserved segments (such as 0xFFFF0~0xFFFFE, for development and testing only).

CCCD — Client Characteristic Configuration Descriptor

CCCD (Client Characteristic Configuration Descriptor) UUID = 0x2902, is the most critical descriptor in GATT, controlling the slave's notification/indication enable/disable:

Master enables notification: Master → CCCD (0x2902) → write 0x0001 → Slave starts pushing Notify

Master disables notification: Master → CCCD (0x2902) → write 0x0000 → Slave stops pushing

CCCD Value	Meaning
0x0000	All notifications/indications disabled
0x0001	Enable notification (Notify)
0x0002	Enable indication (Indicate)

4.4 Notify vs Indicate Comparison

Comparison	Notify	Indicate
Wait for confirmation after sending	✗ No	✓ Yes, wait for application layer ACK
Transmission speed	Fast (no confirmation wait)	Slow (wait for ACK each time)
Reliability	Low (no retransmission on packet loss, except link layer retransmission)	High (application layer confirmation + retransmission)
Typical use cases	High-frequency sensor reporting (heart rate, temperature)	Critical commands (alarms, switch control)

Notify flow: Indicate flow:

Slave —Data→ Master Slave —Data→ Master

(No confirmation, continue sending next) ←ACK—

Slave sends next only after receiving ACK

4.5 BLE Connection Parameters

After a BLE connection is established, the master can request the slave to modify the following connection parameters:

Parameter	Range	Default	Description
Connection Interval	7.5ms ~ 4s	~30ms	Interval between two connection events, determines throughput and power consumption
Slave Latency	0 ~ 499	0	Number of connection events the slave can skip, reducing slave power consumption

Parameter	Range	Default	Description
Supervision Timeout	100ms ~ 32s	~2s	How long without receiving data before determining disconnection

Connection interval 7.5ms: ■■■■■■■■ (high throughput, high power)

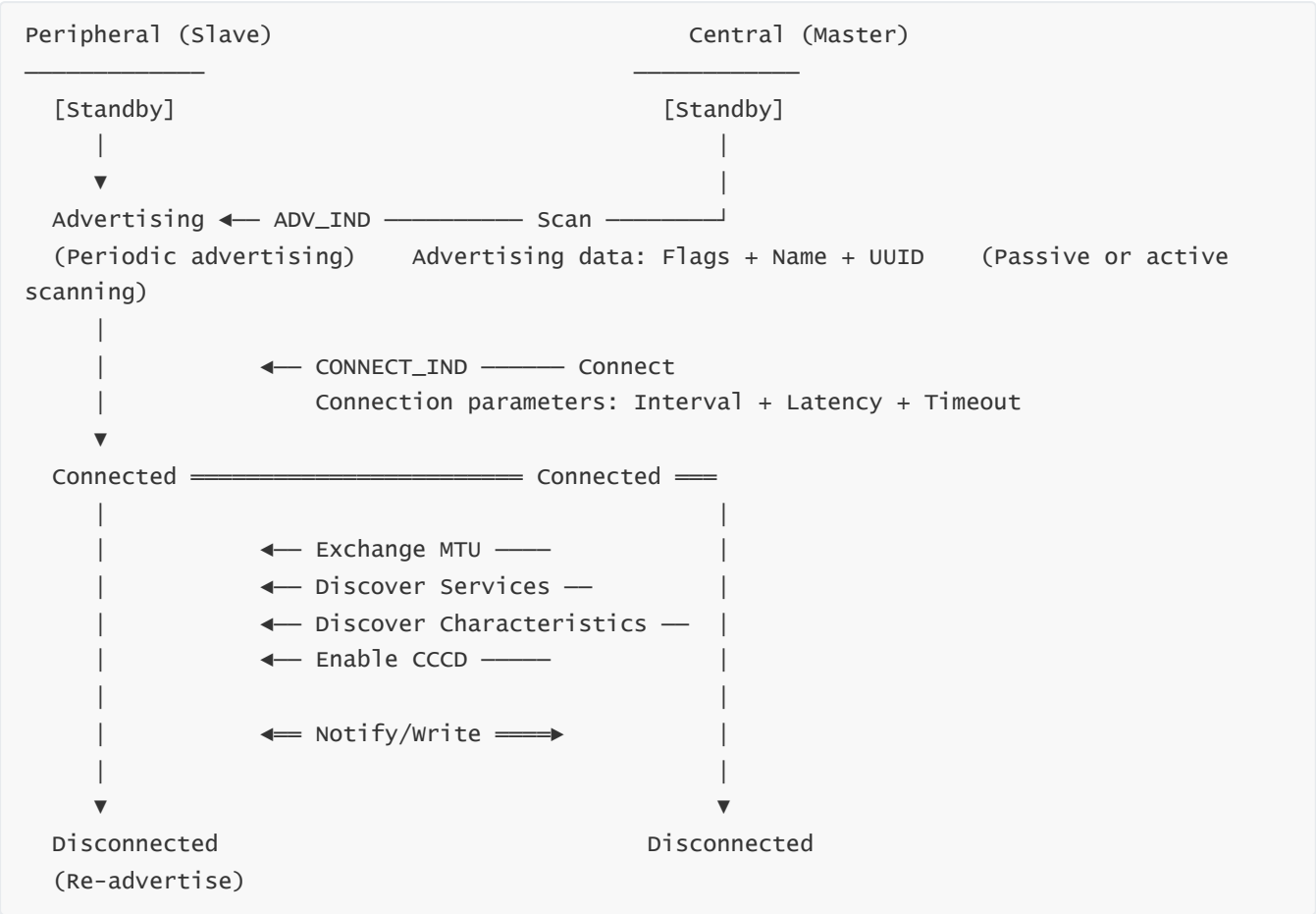
Connection interval 100ms: ■■■■ (medium)

Connection interval 1000ms: ■■ (low throughput, extremely low power)

Connection interval = 7.5ms × N (N ranges from 6 to 3200). Smaller values mean higher throughput but higher power consumption, larger values mean lower power consumption but increased latency. Sensor applications typically use 100ms~1s, while audio applications require ≤30ms.

5. BLE Advertising and Connection Process

5.1 Complete Lifecycle



5.2 Advertising Interval and Discovery Latency

Smaller advertising interval means faster discovery, but higher power consumption:

Advertising Interval	Average Discovery Latency	Average Power Consumption	Use Case
20ms (fast advertising)	~10ms	~1.5mA	Initial pairing/provisioning stage
100ms	~50ms	~0.4mA	Normal connectable state
1s (slow advertising)	~500ms	~0.05mA	Long standby, beacons

Android / iOS have **scan window limitations** - iOS background scanning occurs approximately once every 1~2 seconds, so the advertising interval should not exceed 500ms, otherwise the phone may take a long time to discover the device. Some systems also require the advertising data to contain specific Service UUIDs for the App to filter in the background.

5.3 BLE 5.0 Extended Advertising

BLE 5.0 introduced extended advertising mechanism, significantly increasing advertising data capacity from 31 bytes:

Legacy advertising (BLE 4.x):
3 advertising channels, max 31 bytes payload per packet

Extended advertising (BLE 5.0):
Supports carrying advertising data on 37 data channels (not on advertising channels),
can chain-send multiple packets (max 1650 bytes payload)

Applications: Indoor navigation, asset tracking, electronic price tag batch updates

6. SM — Security Manager

6.1 Pairing Methods

BLE Security Manager (SM) is responsible for pairing and bonding between devices:

Pairing Method	Security	User Interaction	Use Cases
Just Works	Low (no MITM protection)	No interaction required	Sensors without UI (thermometer, humidity meter)
Passkey Entry	Medium	One side displays 6-digit number, other side enters	Keyboard connecting to phone, phone connecting to device with screen
Numeric Comparison	High (LE Secure Connections)	Both sides display 6-digit number, user confirms match	Recommended for BLE 4.2+, phone connecting to peripherals

Pairing Method	Security	User Interaction	Use Cases
Out of Band (OOB)	Highest	Key transferred through external channel like NFC / QR code	Enterprise security, access control systems

6.2 Three Phases of Pairing

Phase 1: Feature Exchange

Both sides exchange IO Capabilities (display/keyboard availability), security requirements
→ Negotiate pairing method (Just works / Passkey / OOB)

Phase 2: Key Generation

STK (Short Term Key, legacy pairing, BLE 4.0/4.1)
or LTK (Long Term Key, LE Secure Connections, BLE 4.2+)
→ Establish encrypted link

Phase 3: Key Distribution

LTK + IRK (Identity Resolving Key) + CSRK (Connection Signature Resolving Key)
→ Bonding complete, subsequent reconnections automatically encrypted (no need to pair again)

Pairing ≠ Bonding. Pairing is a one-time encryption process, bonding stores keys in NVS flash, subsequent connections automatically encrypted - this is what "paired devices list" means in Flutter / Android Apps.

7. Classic Bluetooth Profiles Introduction

While BLE is the mainstream for IoT, ESP32-S3 also supports Classic Bluetooth, which can be used for the following classic Profiles:

Profile	Full Name	Typical Applications	ESP32-S3 Support
SPP	Serial Port Profile	Wireless serial passthrough - simulating UART to replace wired serial	✅ Most commonly used
A2DP	Advanced Audio Distribution Profile	Bluetooth audio streaming (headphones, speakers)	✅
AVRCP	Audio/Video Remote Control Profile	Bluetooth remote control (prev/next track, volume)	✅
HID	Human Interface Device	Bluetooth keyboard/mouse/gamepad	✅
HFP	Hands-Free Profile	In-car hands-free calling	✅

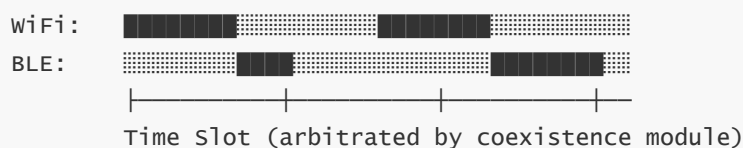
SPP is the most commonly used Profile in ESP32-S3 Classic Bluetooth development - it establishes a virtual serial port between two devices through RFCOMM serial emulation protocol, presenting as byte stream sending/receiving to upper layer code, similar to UART but without physical wiring. Suitable for wireless debugging, passthrough control, data collection and other scenarios.

8. ESP32-S3 Bluetooth Hardware Features

Feature	Parameters
Bluetooth Standards	BLE 5.0 + BR/EDR (Classic)
Operating Frequency Band	2.4 GHz ISM
Max Transmit Power	+20 dBm (Class 1)
Max Receive Sensitivity	-97 dBm (BLE 1M)
BLE Max Throughput	~1.4 Mbps (2M PHY, ~700 kbps application layer)
Classic Bluetooth Max Speed	2.1 Mbps (EDR)
Antenna	On-board PCB ceramic antenna / IPEX external antenna
Protocol Stack	Bluedroid (full) / NimBLE (BLE only)

8.1 Bluetooth and WiFi Coexistence — 2.4GHz Resource Arbitration

Bluetooth and WiFi share the same antenna and 2.4GHz RF front-end - both **cannot transmit simultaneously**. ESP32-S3 resolves conflicts through **Time Division Multiplexing (TDM)**:



Coexistence Strategy	Description
ESP_COEX_PREFER_WIFI	WiFi priority, Bluetooth actively yields when conflicts occur → BLE throughput decreases but WiFi runs smoothly
ESP_COEX_PREFER_BT	Bluetooth priority, WiFi TCP ACK may be delayed → BLE connection stable but WiFi may stutter
ESP_COEX_PREFER_BALANCE	Balanced mode, dynamic arbitration

In practice, if BLE connection interval $\geq 30\text{ms}$ (medium/low throughput scenarios), WiFi and BLE can "peacefully coexist" - BLE occupies scheduled connection event slots (a few milliseconds), WiFi freely uses RF for the rest of the time. At high-frequency BLE (approaching max throughput ~700kbps) or dense WiFi transmission (TCP streaming, OTA), they will interfere with each other, requiring tuning coexistence strategy or staggering usage periods.

8.2 Coexistence Configuration

Configure coexistence strategy in `menuconfig`:

```
Component config → Bluetooth → Bluedroid → Bluetooth Coexistence
```

Or adjust at runtime in code:

```
// Set BT/WiFi coexistence mode to balance
esp_coex_set_preferred_mode(ESP_COEX_PREFER_BALANCE);
```

9. ESP-IDF Bluetooth Protocol Stack Selection

9.1 Bluedroid vs NimBLE

ESP-IDF provides two Bluetooth Host protocol stacks:

Comparison Dimension	Bluedroid	NimBLE
Bluetooth Type	Dual-mode (BR/EDR + BLE)	BLE only
ROM Usage	~512 KB	~32 KB
RAM Usage	~120 KB (minimum)	~4.5 KB
Feature Completeness	Full: SPP, A2DP, HID, GATT full functionality	Lite: GAP + GATT only, BLE 5.0 main features
Configuration Complexity	High, many menuconfig options	Low, initialization ~10 lines of code
Use Cases	Requires Classic Bluetooth (audio, SPP passthrough) or full BLE features	BLE only, code size sensitive scenarios
ESP-IDF Default	☒ Enabled by default	Requires manual selection

9.2 Protocol Stack Selection

```
Need Classic Bluetooth (SPP/A2DP/HID)? —Yes→ Bluedroid (only choice)
|
No (BLE only)
|
└─ Need full BLE features (Mesh, Extended Advertising, LE Audio)?
    |
    └─ Yes→ Bluedroid
    |
    └─ Lite BLE peripheral/central role, code size/RAM sensitive?
        └─ Yes→ NimBLE
```


Subsequent BLE slave and master experiments in this project use **NimBLE** protocol stack. For IoT teaching scenarios requiring only BLE peripheral/central roles, NimBLE's code size and memory overhead is much smaller than Bluedroid, making it a more suitable choice.

9.3 Bluedroid menuconfig Key Configuration

Required configuration when using Bluedroid:

```
Component config → Bluetooth
├─ [*] Bluetooth
├─ Bluetooth Host → Bluedroid
├─ Bluedroid Options
│   ├─ [*] Include GATT
│   ├─ [ ] Include Classic Bluetooth (as needed)
│   └─ [*] Enable BLE 5.0 features (as needed)
```

When using NimBLE:

```
Component config → Bluetooth
├─ [*] Bluetooth
├─ Bluetooth Host → NimBLE
├─ Bluetooth Controller → BLE Only
```

10. Common Troubleshooting

Symptom	Cause	Solution
Crash after Bluetooth initialization	NVS not initialized or partition corrupted	Add try-erase logic to <code>nvs_flash_init()</code> ; or erase entire flash via IDE
Advertising start failed <code>rc=12</code>	BLE controller not enabled	Ensure <code>CONFIG_BT_ENABLED=y</code> in menuconfig, controller mode = BLE Only
Phone cannot scan ESP32-S3	Advertising interval too large / no Service UUID	Reduce advertising interval to $\leq 100\text{ms}$; confirm advertising data contains Flags + Name
Disconnects immediately after connection (0~2s)	Incompatible connection parameters / too far	Move closer ($\leq 5\text{m}$); confirm no strong WiFi co-channel interference
WiFi frequent disconnections during BLE connection	2.4G RF conflict	Adjust coexistence strategy to <code>ESP_COEX_PREFER_WIFI</code> ; or increase BLE connection interval to $\geq 100\text{ms}$
iOS background scan cannot find device	iOS doesn't notify App about advertising without Service UUID	Add specific Service UUID to advertising data (iOS App needs to declare corresponding background mode)

Symptom	Cause	Solution
Cannot find custom service in GATT discovery	NimBLE service table definition error	Confirm <code>ble_gatt_svc_def</code> array ends with <code>{0,}</code> , UUID format correct
Notify cannot be sent	CCCD not enabled / invalid conn_handle	Confirm master has written CCCD = 0x0001; confirm conn_handle matches current connection
<code>ATT write not permitted</code> log	Wrote to characteristic value handle without Write permission	Confirm write operation targets CCCD descriptor handle, not characteristic value handle; or add Write flag to characteristic declaration
Bluetooth interrupted during OTA	Lower layer restart during OTA causes Bluetooth driver reset	After OTA completes, re-run <code>nvs_flash_init()</code> + <code>nimble_port_init()</code>